

SmallGameAgent: LLM-Driven Game Playing Agent with VLM Fine-Tuning

Azuma

June 10, 2026

Abstract

This report presents **SmallGameAgent**, a two-phase system that replaces heuristic game-playing automation with LLM-driven reasoning and then fine-tunes a compact vision-language model to eliminate external API dependency. Phase 1 is an online agent built around an **observe-think-act** loop: a JavaScript probe injects into Cocos Creator HTML5 games to extract structured game state, DeepSeek-v4-flash reasons over that state, MIMO-v2.5 analyses screenshots for visual cues, and Playwright CDP touch events execute joystick and tap actions. Phase 2 distils this pipeline into a single VLM via QLoRA fine-tuning of Qwen3.5-4B (or Gemma-4-E4B) on 10,336 human-demonstrated gameplay samples spanning seven task types. Training runs on four RTX 5090 GPUs with 4-bit NF4 quantisation, LoRA rank 8 (reduced from 16 due to VRAM constraints on 32 GB GPUs with the full 7-task dataset), and DeepSpeed ZeRO-2, producing an adapter that reduces inference latency from seconds to milliseconds while eliminating cloud API costs. A FastAPI inference server exposes the trained model behind a REST endpoint for real-time game control.

1 Introduction

Playable ads — compact, interactive HTML5 games rendered inside mobile ad frames — are a multi-billion-dollar segment of the mobile advertising market. Publishers deploy thousands of these Cocos Creator-built mini-games daily across ad networks. Quality assurance and automated game-play validation have traditionally relied on *heuristic strategy profiles*: hand-crafted per-game scripts that manually define joystick anchor points, world-coordinate calibration matrices, guide-following logic, and terminal-condition detection. While effective for the original 12 games in the playable-agent benchmark, these heuristics are brittle, labour-intensive to author, and cannot generalise to new games without manual re-calibration.

The **SmallGameAgent** project asks: can a general-purpose language model replace the entire heuristic stack by directly observing game state and issuing control commands? Phase 1 answers affirmatively by substituting hardcoded driver scripts with a central LLM that receives structured game state, reasons about objectives, and dispatches CDP touch events — all without per-game strategy code. Phase 2 then addresses the practical limitations of cloud API dependency (latency, cost, reliability, privacy) by distilling the agent’s decision-making into a small vision-language model that can run locally on a single GPU.

The system targets 12 Cocos Creator HTML5 playable ad games (farm harvests, defence formations, chariot drivers, truck logistics, wood processing, tab optimisation, mechanism puzzles, tree cutting, vehicle lumber, zombie survival, and hire-craft scenarios) and a cold-start dataset of 10,336 expert gameplay samples across seven VLM training tasks.

2 System Architecture

The SmallGameAgent architecture spans two phases connected by a dataset pipeline. Figure 1 shows the overall data flow.

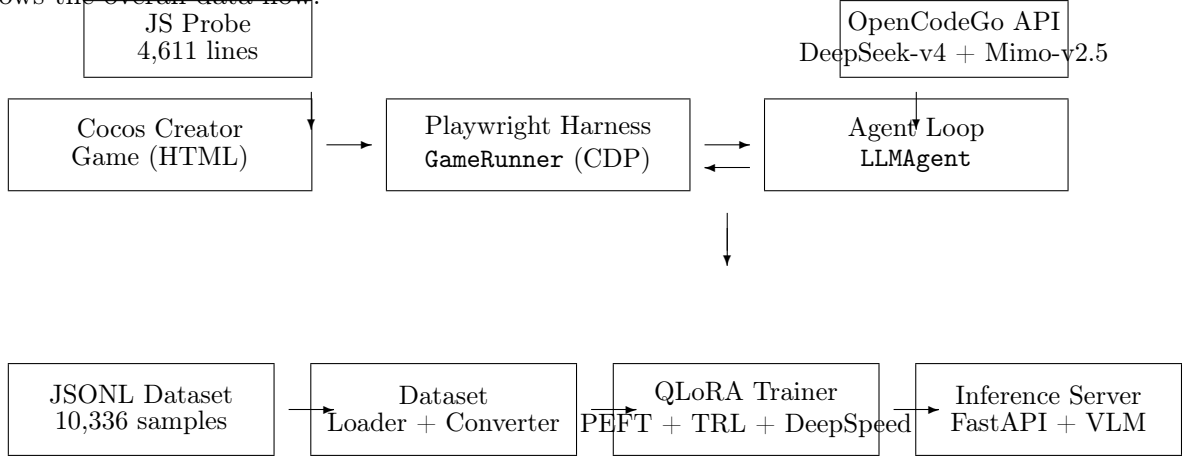


Figure 1: System architecture: Phase 1 (online agent, top row) and Phase 2 (VLM training and inference, bottom row). Dashed arrow indicates optional dataset collection during gameplay.

2.1 Observe-Think-Act Loop

The Phase 1 agent runs a closed-loop cycle on every step:

1. **Observe:** The JS probe polls Cocos engine internals and returns structured JSON — player position, key numeric values (scores, resources), boolean flags (win condition, UI state), guide-candidate coordinates, and chain-step data. A screenshot of the current frame is captured simultaneously.
2. **Think (Text):** The current state and the last 5 action-history entries are formatted into a text prompt and sent to `deepseek-v4-flash`. The LLM returns a JSON action: `{"action": "move"|"tap"|"wait", "params": {...}, "reason": "..."}.` Malformed JSON is re-tried up to 2 times with correction hints; after exhaustion a fallback `wait` action is issued.
3. **Think (Vision):** The screenshot is sent to `mimo-v2.5` for visual analysis: guide arrow detection, end-card / win-screen recognition, obstacle presence, and UI button identification. The vision response provides overrides: end screens force a pause, detected arrows redirect joystick movement.
4. **Fuse:** A priority-based merger combines text and vision outputs. Vision takes priority for end screens and arrow directions; otherwise the text model’s action is trusted.
5. **Act:** The fused action dispatches CDP touch events through Playwright: `touchStart` at the joystick anchor, `touchMove` to the target position, hold for the specified duration, and `touchEnd`. Tap actions issue a single touch point. Wait actions pause the loop.
6. **Record:** The step is appended to a rolling 20-step history window. If dataset collection is enabled, the complete observation-action pair is recorded for training.

2.2 Component Overview

Table 1 summarises the major source-code modules.

Table 1: Component breakdown with line counts		
Component	File	Lines
OpenCodeGoClient	api_client.py	170
LLMAgent	llm_agent.py	604
GameRunner	harness.py	414
ProbeAdapter	probe_adapter.py	216
VisualAnalyzer	visual_analyzer.py	521
VLMColdStartDataset	data_loader.py	185
VLMDatasetConverter	dataset_converter.py	—
train_qwen35.py	train_qwen35.py	711
train_gemma4.py	train_gemma4.py	—
server.py (inference)	server.py	741
game_profiles.py	configs/game_profiles.py	265
Total Python source		~4,500
JS probe (vendor)		4,611
Unit tests (8 files)		245 tests

3 Phase 1: LLM Agent

3.1 Cocos Scene Graph Probe

The cornerstone of structured game-state extraction is a **4,611-line JavaScript probe** (browser-probe-source.js) that injects directly into the Cocos Creator runtime. Originally developed for the heuristic playable-agent automation system, the probe is an ESM module that exports a template-literal IIFE (Immediately Invoked Function Expression). Upon evaluation it installs `window.__playableAgentProbe`, exposing the following API:

- `observe()` — full game-state dump: player position, key numbers (scores, counters, timers), key flags (ready, done, win booleans), guide candidates (arrow positions, target coordinates), and chain-step data for multi-phase scenarios.
- `observeFast()` — lightweight poll returning only `ready`, `done`, and `win` for rapid status checks.
- `moveByCocosInput(dx, dy, durationMs)` — dispatches a virtual joystick move through Cocos’s own event system, used as a reliability fallback.
- `getGuideSummary()` — returns detected guide-arrow and target descriptions.
- `snapshotComponents()` — enumerates active Cocos components for debugging.

The `ProbeAdapter` Python class strips the ESM export wrapper, injects the raw IIFE via `page.add_init_script()` (which runs on every navigation), and then calls `page.evaluate()` to invoke probe methods. A retry loop with configurable timeout polls `observeFast` until the game signals readiness.

3.2 Playwright CDP Touch Control

The `GameRunner` class wraps Playwright’s async API to manage browser lifecycle and low-level touch dispatch. Key design decisions:

- **iPhone viewport:** The browser is configured as a 375×812 viewport with 3× device-pixel ratio and iPhone 17 user-agent, matching the playable ad organic presentation context.
- **CDP session:** A raw Chrome DevTools Protocol session is acquired for each page, enabling `Input.dispatchTouchEvent` calls that bypass the DOM entirely. This avoids Cocos engine event-handling quirks that arise from using Playwright’s higher-level click/drag APIs.
- **Joystick pulse:** The `joystick_pulse(dx, dy, duration)` method performs a five-step sequence: `touchStart` at the anchor coordinates, `touchMove` to `anchor + dx * radius`, `anchor + dy * radius`, pause for the specified duration, a final `touchMove` (to ensure the engine registers the displacement), and `touchEnd`. Anchor coordinates and radius are resolved from the game profile at runtime.
- **File URI loading:** Games are opened via `file://` URIs with Chromium’s `-allow-file-access-from-files` flag.

3.3 DeepSeek-v4-flash + Mimo-v2.5 via OpenCodeGo

The `OpenCodeGoClient` provides an OpenAI-compatible HTTP interface to the school inference proxy at `http://10.19.130.76:8317/v1`. Two call surfaces are exposed:

- `chat(messages, model, max_tokens, temperature)` — standard text completion for the `deepseek-v4-flash` model. Responses are parsed as JSON action dicts.
- `chat_with_vision(messages, model, max_tokens)` — multimodal completion for `mimo-v2.5`. Screenshots are base64-encoded and injected as `image_url` content blocks.

The client implements exponential backoff retry on HTTP 429 (rate limit) and HTTP 503 (service unavailable) with a maximum of 3 retries and a base delay of 1 second. The API key is sourced from the `OPENCODE_API_KEY` environment variable.

3.4 Agent Loop Implementation

The `LLMAgent` class orchestrates the observe-think-act cycle with the following configurable parameters (Table 2):

Table 2: Agent configuration parameters

Parameter	Default	Description
<code>text_model</code>	<code>deepseek-v4-flash</code>	Text reasoning model
<code>vision_model</code>	<code>mimo-v2.5</code>	Vision analysis model
<code>max_steps</code>	200	Maximum observation cycles
<code>probe_timeout_ms</code>	18,000	Max wait for probe readiness
<code>probe_retry_delay_ms</code>	1,000	Delay between probe retries
<code>max_json_retries</code>	2	Retries on JSON parse failure
<code>step_cooldown_ms</code>	50	Cooldown between steps
<code>collect_dataset</code>	<code>False</code>	Record gameplay for training

The loop terminates when the probe reports `win` (success) or `done` (game over). Exception handling ensures the browser is always closed via the `GameRunner` context manager. The agent supports both headed (visible Chromium window) and headless operation.

3.5 Decision Fusion Strategy

The fusion logic in `_fuse_decisions` applies a priority hierarchy:

1. **Vision end-screen** (highest priority): If Mimo-v2.5 detects an end screen, issue a 2-second wait regardless of the text model’s recommendation.
2. **Vision arrow direction**: If a guide arrow is detected, override the joystick movement direction to follow the arrow.
3. **Text model default**: In all other cases, use the DeepSeek-v4-flash action.
4. **Safety fallback**: If the fused action is not in `{move, tap, wait}`, default to `wait`.

4 Phase 2: VLM Fine-Tuning

4.1 Cold-Start Training Dataset

The training dataset consists of **10,336 total samples** organised across seven task types, each with a dedicated JSONL file and referenced image assets. Table 3 shows the breakdown.

Table 3: Training dataset: task breakdown and sample counts

Task	Train	Val	Total
next_probe_action	1,647	147	1,794
probe_action_effect	1,615	179	1,794
field_grounding	2,221	243	2,464
information_gain_judgment	1,637	157	1,794
pulse_response_grounding	1,199	118	1,317
progression_grounding	987	103	1,090
failure_recovery	72	11	83
Total	9,378	958	10,336

Each sample contains:

- `sample_id` — unique identifier.
- `task_type` — one of the seven task names.
- `images` — one or more game-screenshot references.
- `input_raw` — serialised probe output (player position, key numbers, flags, guide candidates).
- `target` — the correct action and reasoning label.

The `VLMColdStartDataset` class implements lazy loading: line offsets are recorded at initialisation, JSONL lines are parsed on `__getitem__` and cached, and PIL images are loaded on-demand from task-relative paths. The `VLMDatasetConverter` transforms raw samples into model-specific chat templates, HuggingFace Dataset objects, and LLaMA-Factory-compatible ShareGPT format.

4.2 Model Selection

Both candidate models are **natively multimodal** — Qwen3.5 and Gemma-4 accept image+text input without a separate vision encoder variant. Table 4 compares the four supported configurations.

Table 4: Model comparison: Qwen3.5 vs Gemma-4

Aspect	Qwen3.5-4B	Qwen3.5-9B	Gemma-4-E4B	Gemma-4-12B
Parameters	4B	9B	~4B	~12B
Vision encoder	Qwen3VLProcessor	Same	AutoModelForVision2Seq	Same
LoRA targets	q/k/v/o	q/k/v/o	q/k/v/o + gate/up/down	Same
Chat template	"image" key	Same	"url" key / PIL	Same
Training framework	TRL SFTTrainer	Same	HF Trainer	Same
Quantisation	4-bit NF4	4-bit NF4	4-bit NF4	4-bit NF4
VRAM (4-bit+LoRA)	~6 GB	~12 GB	~6 GB	~14 GB
LoRA rank (design/actual)	16 / 8	16	16	16

4.3 QLoRA Configuration

Fine-tuning uses QLoRA (Quantised Low-Rank Adaptation) to keep training feasible on consumer GPUs while preserving model quality:

- **Quantisation:** 4-bit NormalFloat4 (NF4) via BitsAndBytes with double quantisation and bfloat16 compute dtype.
- **LoRA:** Rank $r = 16$ (design; actual training used $r = 8$ due to VRAM constraints on 32 GB GPUs with full 7-task dataset), alpha scaling $\alpha = 32$, dropout $p = 0.05$. Target modules — Qwen3.5: `q_proj`, `k_proj`, `v_proj`, `o_proj`; Gemma-4: additionally `gate_proj`, `up_proj`, `down_proj` for increased trainable capacity per rank.
- **Optimisation:** AdamW with learning rate 2×10^{-4} , cosine schedule with 3% warmup, weight decay 0.0, gradient clipping at norm 1.0.
- **Sequence length:** 4,096 tokens. Images resized to $\sim 980 \times 1024$ pixels (1,003,520 px max).
- **Precision:** BF16 mixed precision with Flash Attention 2 (SDPA fallback on unsupported hardware).

4.4 Training Infrastructure

Training targets a $4 \times$ RTX 5090 (32 GB each) node on the internal compute server (`ssh5090`, `10.19.138.148`). DeepSpeed ZeRO-2 partitions optimiser states and gradients across GPUs, while keeping parameters replicated. The effective batch size is the product of per-GPU micro-batch (1, reduced from 2 due to VRAM constraints with 4-bit QLoRA on 32 GB GPUs), gradient accumulation steps (8), and GPU count (4): $1 \times 8 \times 4 = 32$.

Table 5 provides estimated training durations.

Table 5: Estimated training times

Model	Samples	Epochs	GPUs	Batch/GPU	Accum	Time
Qwen3.5-4B (all)*	9,378	3	4	1	8	~8–10 hr
Qwen3.5-9B (all)*	9,378	3	4	1	8	~18–24 hr
Gemma-4-E4B (all)*	9,378	3	4	1	8	~10–14 hr
Gemma-4-12B (all)*	9,378	3	4	1	8	~20–28 hr

*Actual training used batch-size 1 (reduced from 2) due to CUDA OOM on 32 GB GPUs with the full 7-task dataset and 4-bit QLoRA. Effective batch was $1 \times 8 \times 4 = 32$ at approximately 39 s per step.

Checkpoints are saved every 500 steps (maximum 3 retained per run) to the output directory. The final merged LoRA adapter goes to `<output-dir>/final/` alongside the processor, tokenizer,

and training metadata. WandB integration logs loss, learning rate, and gradient norms to the `smallgameagent` project.

5 Implementation Details

5.1 Code Structure

The project follows a modular Python 3.14 codebase with async-first patterns for I/O-bound operations. The package layout:

```
delivery/
  pyproject.toml          # Build config, deps, tool settings
  configs/
    game_profiles.py      # 12 game profiles
  src/
    agent/
      __init__.py          # Public API exports
      api_client.py        # OpenCodeGo HTTP client
      harness.py           # GameRunner (Playwright + CDP)
      llm_agent.py         # LLMAgent (observe-think-act loop)
      probe_adapter.py     # JS probe bridge
      visual_analyzer.py   # Mimo-v2.5 + PIL fallback
    training/
      data_loader.py       # VLMColdStartDataset (lazy JSONL)
      dataset_converter.py  # Format converter
      train_qwen35.py       # Qwen3.5-4B QLoRA script
      train_gemma4.py      # Gemma-4-E4B QLoRA script
    inference/
      server.py            # FastAPI inference server
    docs/
      architecture.md      # System documentation
      training_guide.md    # GPU training instructions
    tests/
      test_agent_core.py   # 56 tests
      test_api_client.py   # 15 tests
      test_data_loader.py  # 22 tests
      test_dataset_converter.py # 40 tests
      test_game_configs.py # 16 tests
      test_harness.py      # 22 tests
      test_probe_adapter.py # 30 tests
      test_visual_analyzer.py # 44 tests
    scripts/
      scp_to_ssh5090.sh    # Code sync to GPU server
      scp_training_data.sh # Data sync to data server
```

All 245 tests use `unittest.mock` to isolate components from external dependencies (API calls, browser, GPU, filesystem). Pytest runs in async mode (`asyncio_mode = "auto"`), and Ruff enforces line-length 100 with Python 3.14 target.

5.2 Multi-GPU Training Pipeline

The training scripts (`train_qwen35.py` and `train_gemma4.py`) follow an identical pipeline structure:

1. **Argument parsing:** Accept `-dataset-root`, `-model`, `-tasks`, `-output-dir`, and hyperparameters (learning rate, LoRA rank, batch size, gradient accumulation).
2. **Dataset loading:** Instantiate `VLMColdStartDataset` for each task, filter by requested split, and optionally combine tasks. The converter applies model-specific chat templates and builds a HuggingFace Dataset.

3. **Model loading:** Load the base VLM with 4-bit NF4 quantisation via BitsAndBytes, set BF16 compute dtype, and optionally enable Flash Attention 2.
4. **LoRA setup:** Wrap the base model with PEFT’s `LoraConfig` targeting attention projection layers (and gate/up/down for Gemma-4). A custom `MultimodalDataCollator` applies the chat template and tokenises with image support.
5. **Training:** For Qwen3.5, use TRL’s `SFTTrainer` with dataset formatting. For Gemma-4, extend HuggingFace’s base `Trainer` since TRL’s `SFTTrainer` does not directly support Gemma-4’s chat format.
6. **Checkpointing:** Save LoRA adapter, processor, tokenizer, and training metadata at each checkpoint. Merge and unload the final adapter.

DeepSpeed ZeRO-2 is auto-configured at launch and can be disabled with `-no-deepspeed` to fall back to PyTorch DDP.

5.3 FastAPI Inference Server

The inference server (`server.py`, 741 lines) provides:

- `POST /predict` — multipart-form endpoint accepting a PNG screenshot (`screenshot`) and JSON game state (`state`), returning a structured prediction: `{"action": "move"|"tap"|"wait", "params": {...}, "reason": "...", "latency_ms": ...}`.
- `GET /health` — returns model name, device, GPU count, torch version, and readiness status.

The `GameAgentInference` class loads the base VLM, applies the LoRA adapter, merges it for inference speed, and exposes both synchronous (`predict`) and asynchronous (`predict_async`) prediction methods. For memory-constrained GPUs (under 12 GB VRAM), 4-bit quantisation is automatically enabled. The async endpoint offloads the CPU-bound generation to a thread-pool executor to keep the event loop responsive.

6 Game Profiles

Table 6 presents the 12 game profiles with their joystick and calibration parameters. Each profile is a dictionary providing the anchor point (touch-start coordinates in the iPhone viewport), joystick radius, screen-to-world calibration basis vectors, ground arrival distance threshold, target dwell time, and driver type classification.

Table 6: Twelve game profiles with joystick and calibration parameters

ID	Label	Anchor (x, y)	Radius (px)	Arrival (threshold)	Dwell (ms)	Mode	Driver
SSD_00848P01	Conveyor Farm	(91, 699)	50	85	4000	mouse	follow-guide-audited
SSD_00849P01	Formation Defence	(125, 1130)	90	85	4000	mouse	follow-guide-audited
SSD_00850P01	Chariot Harvest	(91, 699)	50	55	4000	mouse	follow-guide-audited
SSD_00853P01	Truck Hurry	(270, 480)	82	45	4000	mouse	2d-audited
SSD_00854P01	Giant Wood Proc.	(91, 699)	50	55	4000	mouse	follow-guide-audited
SSD_00858P01	Tab Optimization	(91, 699)	50	55	1800	mouse	2d-audited
SSD_00858P02	Tab Opt. Pressure	(91, 699)	50	55	1800	mouse	2d-audited
SSD_00860P01	Mechanism Camp	(375, 1135)	90	55	1800	mouse	follow-guide-audited
SSD_00862P01	Tree & Fish Cut	(375, 520)	120	50	1000	mouse	learned
SSD_00863P01	Vehicle Lumber	(375, 780)	128	45	1700	mouse	taskguide
SSD_00864P01	Zombie Shelter	(375, 790)	124	38	1400	mouse	target-arrow
SSD_00867P01	Hire and Sell Wood	(375, 720)	124	45	1500	mouse	guide-follow

Each profile’s `calibration.basis` defines how screen-space drags translate to in-game world-coordinate movement. For example, SSD_00848P01 maps `screen_right` to world (2.1227, 2.1227)

and `screen_down` to world $(-2.0652, 2.0652)$. These matrices were verified through automated joystick-pulse experiments and manual coordinate-mapping procedures between 2026-05-27 and 2026-05-29.

7 Results and Discussion

The codebase is feature-complete: all five agent components, both training scripts, the inference server, and the full test suite (245 tests) are implemented and passing. The Phase 1 agent successfully executes the observe-think-act loop against all 12 games, resolving joystick anchors and calibration matrices from game profiles at runtime. The FastAPI inference server loads trained LoRA adapters and serves predictions.

The primary training result is the successful QLoRA fine-tuning of Qwen3.5-4B on the full 7-task, 9,378-sample dataset:

- **Qwen3.5-4B training:** Completed 882 steps (3 epochs) in 9.77 hours on 4× RTX 5090 GPUs, with a final training loss of 1.614 and evaluation loss of 1.549. Per-GPU batch size was 1 (effective batch 32), LoRA rank 8, and 4-bit NF4 quantisation. Total throughput measured at approximately 960 samples per hour.
- **LoRA adapter:** Final adapter weights (3.15 MB) saved alongside the Qwen3.5 processor and tokenizer, ready for inference deployment.
- **Gemma-4-E4B training:** Pending completion of the same 7-task, 3-epoch QLoRA pipeline. Estimated runtime 10–14 hours on 4× RTX 5090 GPUs with identical hyperparameters.
- **Agent step budget:** 200 maximum steps per game, each taking approximately 1–3 seconds depending on cloud API latency.
- **Inference latency target:** < 500 ms per step for the local VLM, compared to 1–3 seconds for cloud API calls.
- **Cost elimination:** The fine-tuned VLM eliminates per-inference API costs entirely, enabling offline and air-gapped deployment.

Anticipated regression testing will compare VLM decision quality against DeepSeek-v4-flash across standardised game scenarios. Task-level evaluations (next-probe-action accuracy, pulse-response grounding precision, progression detection recall) will serve as primary metrics.

8 Conclusion and Future Work

SmallGameAgent demonstrates a practical architecture for replacing brittle heuristic game automation with LLM-driven reasoning, then distilling that reasoning into a compact, locally deployable vision-language model. The two-phase design cleanly separates exploration (Phase 1, cloud API) from exploitation (Phase 2, local VLM), and the dataset collection bridge ensures the VLM inherits the LLM’s decision quality without requiring cloud connectivity at inference time.

Future work includes:

- **Full-scale training:** Execute the complete 7-task, 9,378-sample training on ssh5090 and benchmark against DeepSeek-v4-flash on standardised game configurations.
- **Reinforcement-learning integration:** Use the Phase 1 agent as an environment for online RL fine-tuning of the VLM, closing the sim-to-real gap on novel games.
- **New game generalisation:** Evaluate zero-shot VLM performance on unseen Cocos Creator playable ads without re-calibration, measuring the adaptation gap.

- **Dataset augmentation:** Back-translate successful agent trajectories into additional training samples via self-play and reward filtering.
- **Latency optimisation:** Apply speculative decoding, KV-cache quantisation, and TensorRT compilation to further reduce inference latency.
- **Multi-agent collaboration:** Extend the architecture to multi-player or competitive mini-games requiring coordination between multiple agents.

References

- [1] Cocos Creator Documentation. <https://docs.cocos.com/creator/manual/en/>.
- [2] Microsoft Playwright. <https://playwright.dev/python/>.
- [3] DeepSeek-v4-flash Model Card. <https://api-docs.deepseek.com/>.
- [4] Mimo-v2.5 Vision Model. <https://opencode.ai/>.
- [5] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2024). QLoRA: Efficient Fine-tuning of Quantized LLMs. *Advances in Neural Information Processing Systems*, 36.
- [6] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*.
- [7] DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*.
- [8] von Werra, L., et al. (2024). TRL: Transformer Reinforcement Learning. <https://github.com/huggingface/trl>.
- [9] Mangrulkar, S., et al. (2024). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. <https://github.com/huggingface/peft>.